Übungsblatt 8

Spring Boot, REST-APIs & Persistenz

5430UE Web and Data Engineering

10. Juni 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Beherrschung der Schichtenarchitektur (Controller, Service, Repository) in einer Spring-Boot-Anwendung.
- Umsetzung von REST-Schnittstellen zur asynchronen Kommunikation mit einem Frontend.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (1/14)

In einer vorherigen Übung haben Sie bereits ein eigenständiges Frontend für einen Währungsrechner mit reinem JavaScript (Fetch-API) implementiert, welches die Wechselkurse von einer externen API abgerufen hat. Ziel dieser Aufgabe ist es nun, diese externe Abhängigkeit aufzubrechen und durch ein eigenes, lokales Spring Boot Backend zu ersetzen.

Wir verlagern die Datenhaltung auf unseren eigenen Server: Die Wechselkurse werden als feste (fixe) Kurse in einer relationalen H2-Datenbank hinterlegt. Ihr bereits geschriebenes JavaScript-Frontend wird anschließend so modifiziert, dass es nicht mehr mit der externen API, sondern asynchron mit Ihrer neuen, eigenen REST-Schnittstelle kommuniziert.

Setup-Voraussetzung: Auf Ihrem System muss ein aktuelles Java Development Kit (JDK) installiert und in ihrer IDE eingerichtet sein. Empfohlen wird die aktuelle LTS-Version **Java 25**. Falls Sie bereits ein anderes JDK installiert haben, können Sie auch dieses weiterhin nutzen. Passen Sie die Versionsnummer im folgenden Schritt entsprechend an.



Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (2/14)

1. **Projekt anlegen mit dem Spring Initializr**:
   Rufen Sie den Spring Initializr auf und konfigurieren Sie das Projekt mit folgenden Einstellungen:
   - Project: `Maven`
   - Language: `Java`
   - Spring Boot: `3.4.x` / `3.5.x` (Aktuelle stabile Version)
   - Group: `wde`
   - Artifact: `waehrungsrechner_spring`
   - Packaging: `Jar`
   - Java: **25** Fügen Sie die folgenden drei Dependencies hinzu: **Spring Web**, **Spring Data JPA** und **H2 Database**. Generieren Sie das Projekt, entpacken Sie die ZIP-Datei und öffnen Sie das Projekt in IntelliJ (Auswahl der `pom.xml`) oder einer IDE Ihrer Wahl.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (3/14)

2. **Projektstruktur & Architektur-Workflow**:
   Da wir ein sauberes Schichtenmodell umsetzen, ist die Platzierung der Java-Klassen entscheidend. Alle Klassen müssen unterhalb des Hauptpakets `src/main/java/wde/waehrungsrechner_spring/` in entsprechenden Unterpaketen (`controller`, `service`, `entity`, `repository`, `dto`) organisiert werden. Legen Sie die Packages als Vorbereitung für die folgenden Aufgaben im beschriebenen Ordner an.

   Der Datenfluss sieht wie folgt aus:
   `Frontend (Browser) -> Controller -> Service -> Repository -> H2-Datenbank`

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (4/14)

3. **Entity (Das Datenbankmodell)**:
   Eine Entität repräsentiert eine relationale Tabellenstruktur im Java-Quellcode. Mithilfe des *Object-Relational Mappings* (ORM) via JPA/Hibernate sorgt Spring Data dafür, dass Instanzen dieser Klasse automatisch in Zeilen der zugrundeliegenden Datenbanktabelle transformiert werden.
   - **Schritt 1 (Projektintegration):** Erstellen Sie im Paket `wde.waehrungsrechner_spring.entity` die Klasse `ExchangeRate.java`.
   - **Schritt 2 (Klassendeklaration):** Recherchieren Sie die passenden Spezifikations-Annotationen aus dem `jakarta.persistence`-Paket, um die Klasse als persistente Datenbank-Entität zu deklarieren. Sorgen Sie über eine entsprechende Annotation explizit dafür, dass die zugehörige Tabelle in der H2-Datenbank den Namen `"exchange_rates"` erhält.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (5/14)

- **Schritt 3 (Struktur und Attribute):** Implementieren Sie die drei folgenden Instanzvariablen:
  - `id` (vom Typ `Long`): Suchen Sie nach den korrekten Annotationen, um dieses Feld als Primärschlüssel festzulegen und anzuweisen, dass die ID-Werte von der Datenbank automatisch generiert werden sollen (Auto-Inkrement via `IDENTITY`-Strategie).
  - `currency` (vom Typ `String`): Für den Währungscode (z.B. `"USD"`).
  - `rate` (vom Typ `Double`): Für den dazugehörigen, festen Wechselkurs (z.B. `1.14`).
- **Schritt 4 (Kapselung & Konstruktoren):** Damit das JPA-Framework die Datensätze zur Laufzeit aus der Datenbank lesen und instanziieren kann, benötigt die Klasse zwingend einen standardmäßigen, parameterlosen Konstruktor – implementieren Sie diesen. Generieren Sie abschließend für alle Felder die passenden Zugriffsmethoden (Getter und Setter).

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (6/14)

4. **Repository (Der Datenbankzugriff)**:
   - Importieren Sie die bereitgestellte, lückenhafte Quellcodedatei `ExchangeRateRepository.java` aus Stud.IP in das Paket `wde.waehrungsrechner_spring.repository`. Darin ist das Interface `ExchangeRateRepository` enthalten, welches von `JpaRepository<ExchangeRate, Long>` erbt.
   - Da wir Wechselkurse gezielt anhand ihres eindeutigen Währungscodes (z.B. `"CHF"` oder `"USD"`) aus der Datenbank auslesen wollen, müssen Sie dem Interface eine passende abstrakte Methodensignatur hinzufügen. Nutzen Sie die Namenskonventionen der *Derived Query Methods* (abgeleitete Abfragemethoden) von Spring Data JPA, um den Methodennamen so zu wählen, dass das Framework die zugrundeliegende SQL-Abfrage (`SELECT * FROM exchange_rates WHERE currency = ?`) vollautomatisch im Hintergrund generiert.
   - Um potenziellen Laufzeitfehlern (wie einer `NullPointerException`) vorzubeugen, falls eine angefragte Währung nicht in der Datenbank existiert, ist der Rückgabetyp der Methode als typsicherer Container-Typ (`Optional`) deklariert, welcher die gesuchte `ExchangeRate`-Entität umschließt.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (7/14)

5. **DTO (Das Datentransferobjekt)**: Ein *Data Transfer Object* (DTO) ist ein Entwurfsmuster aus der Softwarearchitektur. Es dient dazu, Daten schichtenübergreifend – in unserem Fall zwischen dem Backend-Server und dem Frontend-Client im Browser – effizient zu transportieren. Sie kapseln die Struktur, die der Client für die Darstellung benötigt, und reduzieren so die Kopplung zwischen Datenbankdesign und API-Schnittstelle.
   - **Schritt 1 (Projektintegration):** Erstellen Sie im Paket `wde.waehrungsrechner_spring.dto` die Klasse `ConversionDTO.java`.
   - **Schritt 2 (Datenstruktur):** Definieren Sie die Klasse als reinen Datencontainer für die JSON-Antwort an das Frontend. Implementieren Sie hierfür die drei benötigten Instanzvariablen für den ursprünglichen Ausgangsbetrag (`amount`), den Währungscode der Zielwährung (`targetCurrency`) sowie das berechnete Endergebnis (`result`). Wählen Sie jeweils den passenden Java-Datentyp.
   - **Schritt 3 (Schnittstellen-Design):** Implementieren Sie einen passenden Konstruktor, der alle drei Felder initialisiert, sowie die entsprechenden Zugriffsmethoden (Getter), damit das Spring-Framework die Daten beim Serialisieren korrekt in ein JSON-Objekt transformieren kann.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (8/14)

6. **Controller (Die REST-Schnittstelle)**: Der Controller fungiert als zentrale Schnittstelle, die HTTP-Anfragen des Clients entgegennimmt und die passenden Antworten liefert.
   - **Schritt 1 (Projektintegration):** Importieren Sie die bereitgestellte, lückenhafte Quellcodedatei `CurrencyController.java` aus Stud.IP in das Paket `wde.waehrungsrechner_spring.controller`.
   - **Schritt 2 (Klassendeklaration):** Versehen Sie die Klasse mit den geeigneten Spring-Annotationen. Kennzeichnen Sie diese so, dass sie als REST-Controller agiert, um eine asynchrone API bereitzustellen, die Daten direkt im JSON-Format serialisiert. Definieren Sie zudem den globalen Basis-Pfad `"/api/currency"` für alle Endpunkte dieser Klasse.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (9/14)

- **Schritt 3 (Endpunkt-Implementierung):** Vervollständigen Sie die lückenhafte Methode `convert` nach folgenden Anforderungen:
  - **Routing:** Annotieren Sie die Methode so, dass sie auf HTTP-GET-Requests unter dem relativen Pfad `/convert` reagiert.
  - **Parameter-Binding:** Nutzen Sie die passenden Annotationen für die Methodenparameter, um die Query-Parameter `amount` (als `Double`) und `targetCurrency` (als `String`) typsicher aus dem URL-Aufruf zu extrahieren.
  - **Service-Aufruf:** Übergeben Sie die extrahierten Parameter an den injizierten `CurrencyService`, um die Wechselkursberechnung durchzuführen, und fangen Sie den Rückgabewert in einem `ConversionDTO`-Objekt auf.
  - **Response-Handling:** Die Methode deklariert als Rückgabetyp ein generisches `ResponseEntity<?>`. Implementieren Sie eine Validierung des Service-Ergebnisses: Liefert der Service `null` (da die Zielwährung in der Datenbank nicht existiert), retournieren Sie eine `400 Bad Request`-Antwort mit dem Fehlertext `"Währung nicht gefunden"`. War die Berechnung erfolgreich, geben Sie das DTO innerhalb einer `200 OK`-Antwort an das Frontend zurück.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (10/14)

7. **Service (Die Geschäftslogik)**: Der Service kapselt die eigentliche Anwendungslogik. Er entlastet die Controller-Schicht, indem er Daten aus den Repositories anfordert, verarbeitet (berechnet) und in das gewünschte Zielformat transformiert.
   - **Schritt 1 (Projektintegration):** Importieren Sie die bereitgestellte, lückenhafte Quellcodedatei `CurrencyService.java` aus Stud.IP in das Paket `wde.waehrungsrechner_spring.service`.
   - **Schritt 2 (Komponentendeklaration):** Welche Annotation wird in Spring Boot benötigt, um eine Klasse als Service-Komponente (eine sogenannte Bean) im Anwendungs-Kontext zu registrieren? Versehen Sie die Klasse mit dieser passenden Annotation, damit sie für das automatische Dependency-Injection-System sicht- und verwaltbar wird.
   - **Schritt 3 (Dependency Injection):** Damit der Service Daten abfragen kann, benötigt er Zugriff auf die Datenbankschicht. Deklarieren Sie das `ExchangeRateRepository` als unveränderliches (`final`) Klassenattribut. Implementieren Sie anschließend die Konstruktor-basierte *Dependency Injection*, um das Repository beim Instanziieren des Service automatisch durch das Framework injizieren zu lassen.



Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (11/14)

- **Schritt 4 (Logik-Implementierung der Methode `convert`):** Vervollständigen Sie die lückenhafte Methode:
  - **Datenabfrage:** Rufen Sie Ihre selbst definierte Abfragemethode des Repositories auf, um den Wechselkurs für die übergebene Zielwährung (`targetCurrency`) zu ermitteln. Fangen Sie das Ergebnis in einem typsicheren `Optional`-Container auf.
  - **Validierung und Fallunterscheidung:** Prüfen Sie mithilfe der Methoden des `Optional`-Objekts, ob ein passender Datensatz aus der Datenbank geliefert wurde.
  - **Berechnung und Rückgabe:**
    - Falls kein Kurs zur Währung gefunden wurde (das Objekt ist leer), soll die Methode direkt `null` zurückgeben.
    - Falls ein Kurs existiert, extrahieren Sie das `ExchangeRate`-Objekt, multiplizieren Sie den Ausgangsbetrag (`amount`) mit dem ausgelesenen Wechselkurs (`rate`) und verpacken Sie alle Daten in ein neu instanziiertes `ConversionDTO`, welches anschließend zurückgegeben wird.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (12/14)

8. **Anwendungskonfiguration & Datenbank-Initialisierung**: Die Datei `application.properties` dient als zentrale Konfigurationsdatei einer Spring-Boot-Anwendung. In ihr werden globale Verhaltensweisen des Frameworks gesteuert, wie etwa Server-Ports, Verbindungseinstellungen zu externen Systemen oder die detaillierte Einrichtung von Datenbanken.
    - **Projektintegration:** Öffnen Sie die Datei `src/main/resources/application.properties` und kopieren Sie die Inhalte aus der zur Verfügung gestellten `application.properties`-Datei hinein.
    - **Datenbank-Einstellung:** Durch diese Vorgaben wird die H2-Datenbank als dateibasierte Datenbank eingerichtet (sodass Testdaten auch nach einem Neustart erhalten bleiben), die grafische Weboberfläche der H2-Konsole aktiviert und Hibernate angewiesen, die Tabellenstrukturen bei jedem Anwendungsstart automatisch auf Basis der Java-Entities neu zu generieren (`create`).

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (13/14)

Die Zeilen `spring.jpa.defer-datasource-initialization=true` sowie `spring.sql.init.mode=always` weisen Spring Boot an, die automatische Tabellengenerierung durch Hibernate vollständig abzuwarten, bevor externe SQL-Skripte ausgeführt werden.

- Erstellen Sie im Ordner `src/main/resources/` die Datei `data.sql`. Fügen Sie dort `INSERT INTO`-Statements ein, um die Tabelle `exchange_rates` beim Start standardmäßig mit festen Kursen für `'USD'` (z.B. 1.14) und `'CHF'` (z.B. 0.93) zu befüllen. Die Spaltenbenennungen müssen mit den Attributnamen der Entity-Klasse übereinstimmen.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank (14/14)

9. **Einbindung und Anpassung des alten Frontends**: Zuletzt verknüpfen wir Ihr bereits existierendes Frontend mit dem neuen Backend.
   - Kopieren Sie Ihre alten Frontend-Dateien (`index.html`, `script.js` und `style.css`) in das Verzeichnis `src/main/resources/static/`. Spring Boot liefert Dateien aus diesem Verzeichnis automatisch als statische Ressourcen aus.
   - Öffnen Sie die `script.js` und passen Sie die `fetch()`-Funktion an. Ändern Sie die Ziel-URL so ab, dass sie nun auf Ihren lokalen Spring Boot Endpunkt verweist: `/api/currency/convert?amount=${amount}&targetCurrency=${target}`.
     Da das resultierende JSON direkt die drei benötigten Werte enthält, kann auf die Konvertierung in `convertedValue` verzichtet werden.
   - Starten Sie die Spring Boot Anwendung über die Hauptklasse. Rufen Sie im Browser `http://localhost:8080` auf und testen Sie, ob die Berechnungen nahtlos mit den Werten aus Ihrer eigenen H2-Datenbank durchgeführt werden.

Eigenes Währungsrechner-Backend mit Spring Boot und H2-Datenbank — Lösung

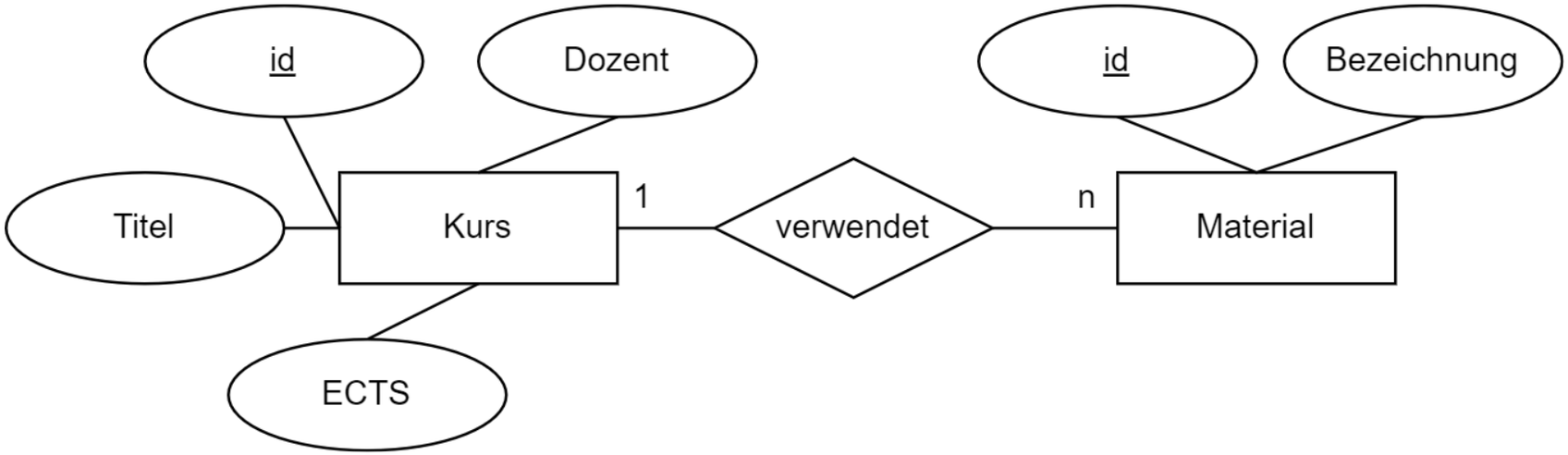
Siehe Projekt `waehrungsrechner_spring` (umgesetzt mit Java 25, Spring Boot 3.5.14 und integrierter H2-Datenbankanbindung).

Entity-Mapping und Repository



Entity-Mapping und Repository (1/2)

Wir betrachten die einfache Modellierung eines Kurssystems:



Es soll gelten:

- `id`: automatisch generiert
- `Titel`: nicht null, max. 200 Zeichen
- `ECTS`: Ganzzahl, 1–10
- `Dozent` und `Bezeichnung`: String
- `Materialien`: soll eine Liste an Materialien sein, die im Kurs verwendet werden.

Entity-Mapping und Repository (2/2)

1. Modellieren Sie die JPA-Entities `Kurs` und `Material`. Achten Sie auf die korrekte Verwendung der JPA-Annotationen.
2. Erstellen Sie ein passendes `KursRepository`. Implementieren Sie darin eine Methode unter Nutzung der Spring Data JPA Namenskonventionen, die alle Kurse eines bestimmten Dozenten findet und zurückgibt.



Entity-Mapping und Repository — Lösung (1/3)

a) **Entity Kurs:**

```java
@Entity
@Table(name = "kurse")
public class Kurs {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(nullable = false, length = 200)
  private String titel;

  @Min(1) @Max(10)
  private int ects;

  private String dozent;

  @OneToMany(mappedBy = "kurs", cascade = CascadeType.ALL)
  private List<Material> materialien = new ArrayList<>();
}
```

Entity-Mapping und Repository — Lösung (2/3)

Entity Material:

```
@Entity
@Table(name = "materialien")
public class Material {

  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  private String bezeichnung;

  @ManyToOne
  @JoinColumn(name = "kurs_id")
  private Kurs kurs;
}
```



Entity-Mapping und Repository — Lösung (3/3)

b) **Repository:**

```
public interface KursRepository extends JpaRepository<Kurs, Long> {

  /**
  * Findet alle Kurse anhand des Dozentennamens.
  * Spring generiert hierfuer automatisch das SQL:
  * SELECT * FROM kurse WHERE dozent = ?
  */
  List<Kurs> findByDozent(String dozent);

  // alternativ mit JPQL:
  @Query("SELECT k FROM Kurs k WHERE k.dozent = :name")
  List<Kurs> findByDozentName(@Param("name") String name);
}
```



Materialien zum Übungsblatt

- Vorlagendateien Währungsrechner (`Vorlagendateien_waehrungsrechner_spring.zip`)
- Lösung Währungsrechner (`waehrungsrechner_spring_lsg.zip`)